

23070521054- Gauri Gulhane

```
# =====
# PRACTICAL 2
# Housing Price Prediction using Deep Feed Forward Network
# =====

import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error

import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense, Input

# -----
# 1. Load Dataset
# -----
print("Loading Housing Dataset...")

data = fetch_california_housing()

X = data.data
y = data.target

print("Dataset Shape:", X.shape)

# -----
# 2. Split Dataset
# -----
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

# -----
# 3. Normalize Data
# -----
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

print("Data preprocessing completed.")

# -----
# 4. Create Deep Feed Forward Network
# -----
model = Sequential([
    Input(shape=(X_train.shape[1],)),

    Dense(128, activation='relu'),
```

```

        Dense(64, activation='relu'),
        Dense(32, activation='relu'),

        Dense(1)
    ])

# -----
# 5. Compile Model
# -----
model.compile(
    optimizer='adam',
    loss='mse',
    metrics=['mae']
)

print("\nModel Architecture:")
model.summary()

# -----
# 6. Train Model
# -----
history = model.fit(
    X_train,
    y_train,
    validation_split=0.2,
    epochs=30,
    batch_size=64,
    verbose=1
)

# -----
# 7. Prediction
# -----
y_pred = model.predict(X_test, verbose=0).flatten()

# -----
# 8. Evaluation
# -----
mae = mean_absolute_error(y_test, y_pred)

mse = mean_squared_error(y_test, y_pred)

rmse = np.sqrt(mse)

print("\n=====")
print("MODEL EVALUATION")
print("=====")
print("MAE :", round(mae, 4))
print("MSE :", round(mse, 4))
print("RMSE :", round(rmse, 4))

# -----
# 9. Plot Training Loss
# -----
plt.figure(figsize=(8, 5))

plt.plot(
    history.history['loss'],
    label='Training Loss'
)

plt.plot(

```

```
plt.plot(
    history.history['val_loss'],
    label='Validation Loss'
)

plt.xlabel("Epoch")
plt.ylabel("MSE Loss")
plt.title("Training and Validation Loss")

plt.legend()
plt.show()

# -----
# 10. Actual vs Predicted
# -----
plt.figure(figsize=(7, 6))

plt.scatter(
    y_test,
    y_pred,
    alpha=0.4
)

plt.xlabel("Actual House Price")
plt.ylabel("Predicted House Price")

plt.title("Actual vs Predicted House Prices")

plt.show()
```

